ENGINEERING OVERVIEW

Hacking on Daemonarchy

Five repositories, one build-and-release loop, and where Literate AI helps and does not yet.

Edition for 1.5.1. Describes main after the published 1.5.0 release; 1.5.1 items are plans. Commands and gate results were run or read from the repository, not assumed.

ORIENTATION

What this deck answers

Four questions a new contributor asks.

How is it put together?

- Which repository holds what.
- How they are pinned and released together.
- Where a given change goes.

How do I hack on it?

- The Daemonarchy way versus `make buildworld installworld` and `make install` in a port.
- The inner loops by kind of change.
- How to test, and what a test does not prove.

What does Literate AI do here?

- Authority gates, component locks, the test receipt, the retained-source baseline and the release check.
- What each gives you today and what it does not.

How do I contribute?

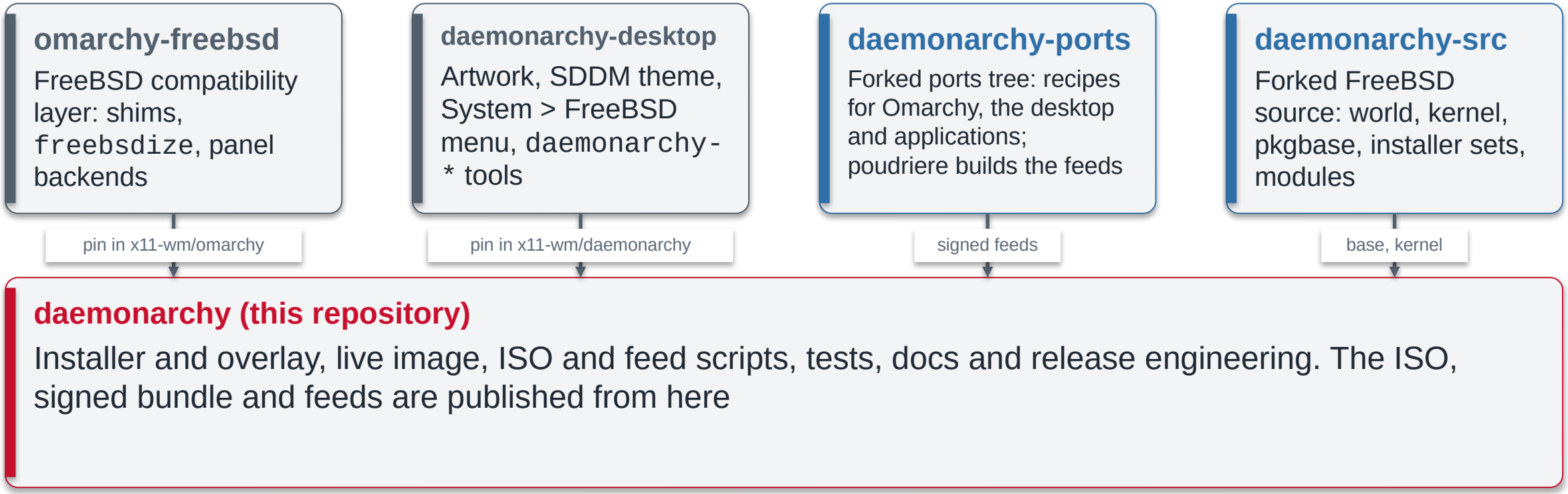
- Branch, pull request, review.
- Evidence and honesty rules.
- Where the open work is.

Source: HACKING.md; SKILL.md; docs/README.md

ARCHITECTURE

Five repositories make Daemonarchy

The product is composed from pinned commits of repositories with distinct jobs.



- Selected by full commit, not branch tip, and held fixed through qualification. A release record names every input and every asset's size and SHA-256.

Source: HACKING.md (The repositories that make up Daemonarchy; How the pieces are pinned)

ARCHITECTURE

Where a change goes

Change	Repository
Artwork, login theme, System > FreeBSD menu, `daemonarchy-*` tools	daemonarchy-desktop, then a pin bump in `x11-wm/daemonarchy`
Installer logic, overlay, live image, ISO and feed scripts, tests, docs	daemonarchy
Package recipes, port patches, new or updated ports	daemonarchy-ports
Kernel, base system, drivers in the base tree	daemonarchy-src
Omarchy's FreeBSD adaptation and compat shims	omarchy-freebsd, then a pin bump in `x11-wm/omarchy`

- A desktop fix reaches users only after a pin bump in daemonarchy-ports and a new package in the feed.

Source: HACKING.md (Where a change goes)

Two ways to build FreeBSD, and where Daemonarchy sits

The upstream commands still describe how FreeBSD is built. Daemonarchy adds pins, signed feeds and evidence around them.

The FreeBSD way

- `cd /usr/src && make buildworld installworld` builds and installs the base system.
- `cd /usr/ports/<category>/<port> && make all install` builds and installs one port.
- Fast, local, and the reference for how upstream works. Nothing records what exactly was built or tested.

The Daemonarchy way

- Work against pinned forks: `daemonarchy-src` and `daemonarchy-ports` selected by full commit.
- Build ports in a poudriere jail made from the release snapshot, then collect and sign them as a feed.
- Build the image with `./build-iso.sh`, test it with `./test-qemu.sh` or `./test-bhyve.sh`, then record the exact inputs.

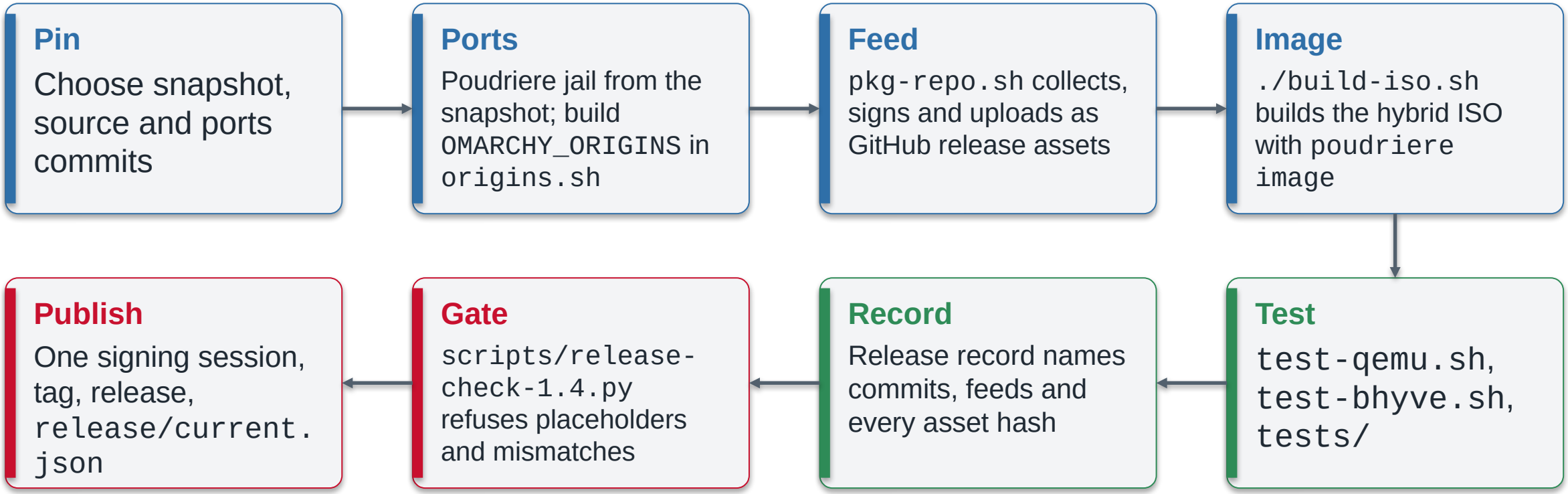
Where they meet

- The installer can lay down shallow `/usr/src` and `/usr/ports` checkouts pinned to the release.
- `daemonarchy-dev-setup` clones the trees into `~/src` so a Daemonarchy machine can develop Daemonarchy.
- Upstream FreeBSD and Omarchy fixes go upstream independently of releases.

Source: HACKING.md (Working on Daemonarchy from Daemonarchy, Setting up a build host); README.md (Optional live checkouts)

The build and release loop

Each stage has a script or a test, and each hands exact inputs to the next.



- Package-only fixes between images bump `PORTREVISION` and ship through the feed; installer changes need a new image.

Source: HACKING.md (Setting up a build host, Testing, Releasing); pkg-repo.sh; build-iso.sh

Inner loops by kind of change

You are changing	Fast loop	Needs
The installer (`overlay/`, Bash helpers)	`./build-iso.sh`, then an unattended `test-qemu.sh` run; the qualified package set can be reused	A new image
A port or its patch	`make extract`, edit, `sudo make reinstall` `clean` in the port directory; poudriere for the real feed	The ports tree
Omarchy itself	`daemonarchy dev link ~/src/omarchy` and `daemonarchy dev sync --watch` rebuild a FreeBSD tree under `~/.local/share/omarchy-dev`	A Daemonarchy session
The FreeBSD layer or the desktop	Lay your checkout over the port's extracted sources with `rsync`, then `make reinstall`	The ports tree
Artwork and menu	Edit in daemonarchy-desktop; a pin bump ships it	A port pin bump

Source: HACKING.md (Changing things, Working on Daemonarchy from Daemonarchy)

DELIVERY

Packages, feeds and signing

Daemonarchy publishes what pkg.FreeBSD.org does not, signed, as GitHub release assets.

Feeds

- Application feed `packages-16-<arch>-X.Y.Z` and checkout feed `checkouts-16-<arch>-X.Y.Z`; published tags are immutable, so a suffix such as `-r2` marks a re-roll.
- 1.5.0 used `packages-16-amd64-1.5.0-r2` and `packages-16-aarch64-1.5.0-r2`.
- The ISO keeps its own packages as a fallback repository, `Daemonarchy-ISO`.

Trust

- The public key ships in the installer overlay as `daemonarchy.pub`.
- Boot files and modules are signed too (amd64 Secure Boot bundle).
- A feed built for a newer `OSVERSION` is refused on an older base.

Signing discipline

- One signing session per release: freeze every signed input, finish QA on unsigned artifacts, then sign in key windows.
- No key on a builder; agents never hold or move key material.
- Keys, private worker evidence and generated images stay out of Git.

Source: HACKING.md (Packages, Releasing, the upgrade model); docs/mastering-1.5.0.md

COMPONENTS

The installer in one slide

Bash helpers under `overlay/usr/local/libexec/daemonarchy-installer/`, driven from root's login on the live image.

Flow

- `launch` starts from root's login.
- `configurator` asks the questions; it vendors Omarchy's own setup form.
- `install` partitions, extracts the base system, writes config, then `pkg` installs the desktop.
- `dashboard` draws the install screen from the install log.

Safety

- `install-integrity` checks package files and dependencies before success is recorded.
- A failed install is retained with its rescue evidence; a console rescue menu is offered.
- Whole disk, selected GPT partition or free space; unattended through a `CIDATA` drive.

Extensions

- A visual SDK (API 1.5) with sandboxed Lua extensions, including a first fish-tank extension.
- LIMIT Built and tested on a build host, not yet on a real image.
- Install trace records every operation for baselines.

Source: HACKING.md (Where things live); README.md (The installer); docs/release-1.5.0.md; docs/install-trace.md

NATIVE PACKAGE PLAN

Resolving packages before installing them

`scripts/native-package-plan.py` compares a real pkg dry-run with one selection from a pinned catalogue graph.

- It copies only pinned repository metadata into a fresh private case and runs `pkg install -nUy` against an empty private database.
- It checks every proposed package version and repository against the graph; any extra action, omission or drift refuses.
- **VERIFIED** The development checkpoint binds 16 passing controller and native FreeBSD checks, and desktop and NVIDIA plans matching all 952 and 960 package identities.
- **LIMIT** Unsigned, local frozen metadata. No installation, no repository authentication, no Literate AI receipt. Signed acquisition and parallel installation remain under PKG-001.

Source: docs/native-package-plan.md; verification/native-package-plan-development.json

PART TWO

Literate AI

What the framework does in this repository on 1.5.x, and what it does not do yet.

WHAT LITAI IS HERE

Literate AI's role in 1.5.x: authority first, source retained

The project is adopted in place. Specifications and evidence live beside the original source; the original source remains implementation and release authority.

What is in the tree

- Nine goals (G1 to G9) and eight Component contracts, declared in `literate.project.json`, with an acceptance mapping.
- A `components/` directory per Component: contract, lock and acceptance files.
- A work queue with stable IDs, and release records.

How it is used

- `litai project validate .` checks authority; `litai verify` reports every gate.
- `litai work record` creates stable work IDs; `SKILL.md` and `AGENTS.md` steer agents.
- A native release gate, `scripts/release-check-1.4.py`, is the one `litai release check` runs.

What it is not (yet)

- Not the build executor: BUILD-001 and PERF-001 moved to 1.5.1 and no speed-up is claimed.
- Not a generator: nothing was regenerated or relocated; no conversion was applied.
- No framework 1.4.0 release exists for the deferred native goals.

Source: PROJECT.md (Completeness); SKILL.md; docs/roadmap/release-1.5.1.md

WHAT LITAI IS HERE

The gates, as `litai verify` reports them today

Gate	Result	What it means
authority	**pass**	Project authority and the documentation review: declared roots, links that resolve, a recorded authority marker.
locks	**pass**	8 committed Component locks are current: a changed contract with a stale lock fails.
receipt	**FAIL**	`verification/current.json`, the native project test receipt, has not been committed. It is reported, never synthesized.
source-intelligence, html	**skipped**	Provider `none`; no declared HTML artifacts.

- Result of `litai verify .` with Literate AI 1.4.0 (development source) on main after 1.5.0. A documentation change makes the authority marker stale until the owner re-records it.

Source: output of litai verify . on the 1.5.1 documentation branch; HACKING.md; docs/architecture/design-traceability.md

ADVANTAGES

What litai gives a contributor

Each advantage is something you can run or read in the repository today.

Intent you can check

- Goals map to Components and acceptance in the design-traceability map, so a change names the contract it touches.
- Locks catch contract drift; the documentation gate catches dead links and out-of-root links.

Evidence that cannot be faked

- Release records bind commits, feeds and every asset hash.
- The release gate refuses placeholders, another version or commit, and asset mismatches.
- A missing receipt stays a failing gate; no passing result is synthesized from logs.

Native work on real workers

- A retained native checkpoint ran 69 checks with 0 failures on a native FreeBSD worker.
- It is a development checkpoint for DEV-001, not a release receipt.

Agents with guardrails

- `SKILL.md` and `AGENTS.md` tell an agent to preserve published tags and bytes and keep keys and private worker addresses out of Git.

Source: SKILL.md; docs/architecture/design-traceability.md; verification/native-retained-acceptance.json; scripts/release-check-1.4.py

LIMITS

What litai does not do yet

Stated plainly so nobody builds a plan on a claim the repository does not make.

Not delivered in 1.5.0

- Framework-executed native components: COMP-001, PERF-001, PERF-002 and DEV-001 moved to 1.5.1 (DEV-001 has 5 of 12 evidence boxes ticked, COMP-001 4 of 11, the others none).
- BUILD-001, the 10x build target: 0 of 5 evidence boxes; no speed-up is claimed.
- The native halves of ADOPT-002, SRC-001 and RELEASE-001 are ADOPT-003, SRC-002 and RELEASE-002.

Planned for 1.5.1

- PLANNED Retained native execution and release enforcement (ADOPT-003, RELEASE-002).
- PLANNED Source and ports as complete Components with per-unit pkg provenance (COMP-001, SRC-002).
- PLANNED Worker-coordinated builds across the repositories (PERF-002, DEV-001).

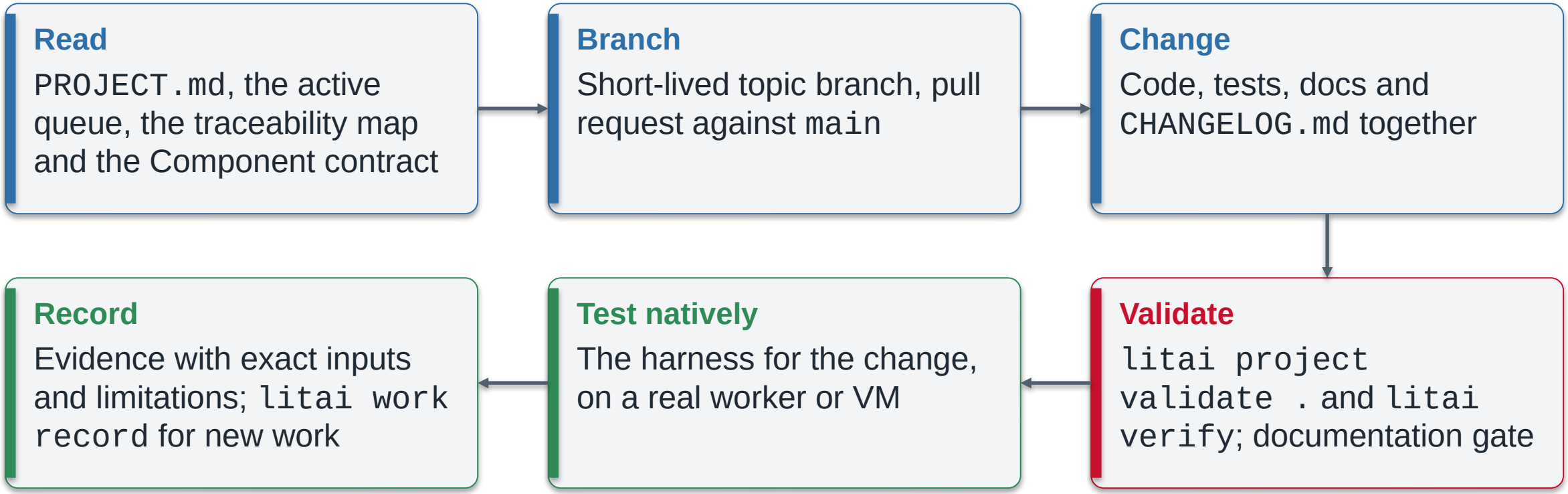
Later

- PLANNED LITAI-PKG-001: ship Literate AI as a port and package so users run source and ports operations with `litai`. Targeted after 1.5; proposed 1.6.0, not scheduled.

Source: docs/roadmap/release-1.5.1.md (Deferred from 1.5.0)

A change, start to finish

What a contributor does, and which gate watches each step.



- A maintainer merges after CI. Documentation edits leave the authority marker stale; the owner re-records it.

Source: SKILL.md; HACKING.md (Contributing, Literate AI project authority)

VERIFICATION

Testing, and what a test does not prove

Harnesses

- `test-qemu.sh <iso>` anywhere QEMU runs, no root: logs in over SSH and checks from inside.
- `test-bhyve.sh <iso>` on a FreeBSD host with VT-x or AMD-V, as root.
- `tests/` holds shell and Python checks for disk layout, install integrity, the installed desktop, release sync and the documentation gate.

Honesty rules

- SSH checks do not qualify a graphical desktop.
- VM results do not stand in for physical GPU, firmware, radio or retained-OS qualification.
- An amd64 result is not an aarch64 result.

Run before a pull request

- `python3 tests/docs-release-sync.py -v` keeps current-state documents in step with `release/current.json`.
- `litai project validate .` for authority; the relevant harness for the code you touched.

Source: HACKING.md (Testing); tests/docs-release-sync.py; docs/release-1.5.0.md

DIAGNOSTICS

When the hardware misbehaves

Tools that exist so that a machine failing early still produces evidence.

Installer side

- Remote agent debug: console-approved pairing on an IP and port, with approval that survives reboot until revoked.
- Rescue console with logs, a shell and remote debugging, gated by the install password.

Boot side

- The hardware bring-up trace image records each startup step in UEFI NVRAM and queues the next experiment unattended; amd64 and aarch64 images exist.

Timing

- Install trace and login trace give per-operation timings; baselines are retained in `verification/`.
- The 1.5.0 login-speed claim is one machine, one first-version patch run.

Source: docs/remote-agent-debug.md; docs/rescue-console.md; docs/boottrace.md; docs/install-trace.md; docs/login-trace.md

OPEN WORK

aarch64: where a developer can help

Item	State
Image	Community validation: installs and boots in QEMU (e1000e install of 919 archives in about 20 minutes) and reaches the SDDM greeter and session in software with VirtIO GPU.
virtio-net receive stall	With QEMU's default virtio-net the guest stops servicing receive on `vtnet0` about a minute into the package fetch. Fixed by a full barrier around ring notification checks, daemonarchy-src PR #59, shipping in 1.5.1.
RTX Spark laptop	Deferred. The stock arm64 kernel stops after `Starting CPU 4 (400)`; `kern.smp.disabled=1` reaches init; a draft bounded-wait fix then panics. No physical-desktop evidence.
Not verified	Hardware 3D, physical GPU, sound, Wi-Fi, upgrade and the live desktop on real hardware.

Source: docs/aarch64.md; docs/release-1.5.0.md; docs/application-note-rtx-spark.md; daemonarchy-src PR #59

FORKABILITY

Forking, today and in 1.5.1

Possible with code edits, not yet easy. FORK-001 plans one configuration and a guide.

What a fork must do today

- Fork the repositories; keep ports pinned to your source and desktop commits.
- Make your own signing key (`pkg-repo.sh keygen`) and Secure Boot bundles.
- Replace hard-coded `jordanhubbard` URLs in the installer, scripts and two ports.
- Have a builder, publish feeds, build and test.

What breaks if you only rename

- The installer refuses its own feed URL.
- The image points at our feeds and checkouts.
- `verify-release-assets.py` checks our repository.

1.5.1

- **PLANNED** FORK-001: one configuration, a fork guide and an example fork CI.
- **NOT VERIFIED** No end-to-end fork by someone else has been tried.

Source: HACKING.md (Forking Daemonarchy); docs/roadmap/release-1.5.1.md

JOIN IN

How to contribute

There is no CONTRIBUTING file; this is the practice from the repository history.

- **Branch and pull request.** `main` is the trunk in every repository: short-lived topic branch, pull request against `main`, CI must pass, a maintainer merges. Never commit to a release branch.
- **Keep secrets out.** No signing keys, private worker evidence or generated images in Git.
- **Evidence over assertion.** State what you ran, on what, with which inputs, and what remains untested.
- **Send upstream what is upstream.** FreeBSD and Omarchy fixes go there, independently of Daemonarchy releases.
- **Pick work from the queue.** `docs/roadmap/active-work.md`, the 1.5.1 plan, and aarch64 reports from real hardware.

Source: HACKING.md (Contributing); docs/roadmap/release-1.5.1.md; docs/aarch64.md (Reporting)